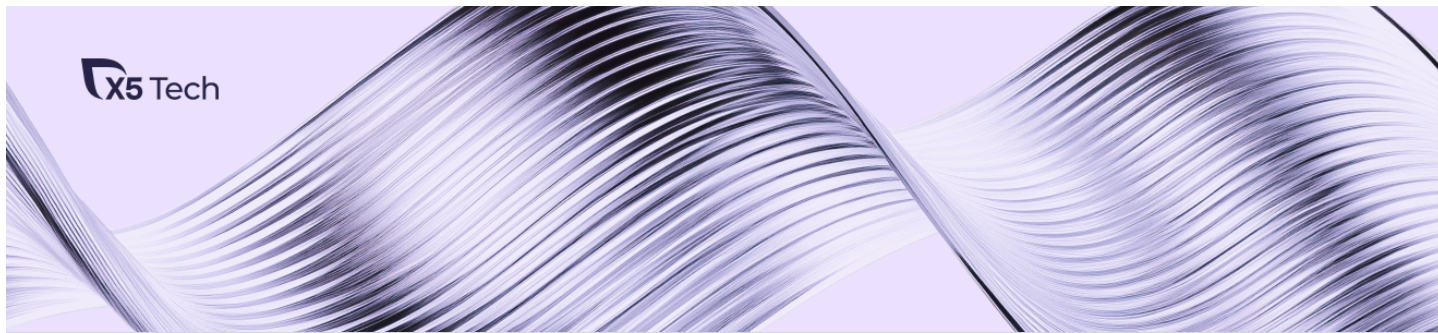




xufana

9 авг 2024 в 10:18

Интеграция LLM в корпоративные чат-боты: RAG-подход и эксперименты



Простой



11 мин



16K

Блог компании X5 Tech, Python*, Искусственный интеллект, IT-компания



Технотекст 7

Всем привет! На связи команда AI-Run из X5 Tech, мы занимаемся генеративными сетями в целом и языковыми моделями в частности. В этой статье мы опишем наш опыт работы с большими языковыми моделями (LLM), их внедрение для обработки корпоративных данных, а также поделимся нашими результатами и выводами.

Ещё мы расскажем о нашем подходе к использованию LLM, подробно остановимся на методе Retrieval Augmented Generation (RAG) и рассмотрим примеры использования чат-ботов на корпоративных порталах X5.

Эта статья будет полезна разработчикам, которые интересуются внедрением LLM для работы с корпоративными данными. Она основана [на нашем выступлении](#) на митапе, но не ограничивается им, а, скорее, дополняет его.

Что такое LLM?

LLM или большая языковая модель (Large Language Model) — это нейросетевая модель, обученная на огромных массивах текстовой информации для понимания и генерации естественного языка. Основная задача LLM — принимать на вход текст и отвечать на него также текстом. Важно отметить, что LLM ограничены длиной принимаемой информации (контекста) — максимального количества токенов, которое способна обработать модель за один вызов.

Токен – языковая единица, с которой работает модель. Чаще всего в токене содержатся 2-4 буквы. Контекст у моделей обычно имеет фиксированную длину, которая обычно составляет 4000 токенов. Например, субтитры нашего часового выступления занимают примерно 8000 токенов.

Работая с LLM, мы сталкиваемся не только с ограничениями на длину контекста: обработка корпоративных данных также накладывает свои ограничения. Пользоваться моделями, которые-hostятся не на наших серверах, мы не можем из-за риска утечки чувствительной информации, а потому нам нужно либо маскировать эти данные, либо пользоваться моделями с открытым кодом и поддерживать их на своих серверах. Сегодня будем говорить о том, как мы работали со вторым вариантом.

Дело в том, что не все LLM "из коробки" идеально подходят для использования. Многие модели имеют недостаточную поддержку русского языка или не обладают специализированными знаниями, необходимыми для определённых задач. Опенсорсные модели также могут дублировать информацию и склонны к галлюцинациям, что снижает качество ответов.

Для решения этих проблем можно дообучать модели на собственных данных, но это часто бывает сложно и дорого. Как же тогда быть? Здесь на помощь приходит метод Retrieval Augmented Generation (RAG).

RAG: альтернатива дообучению

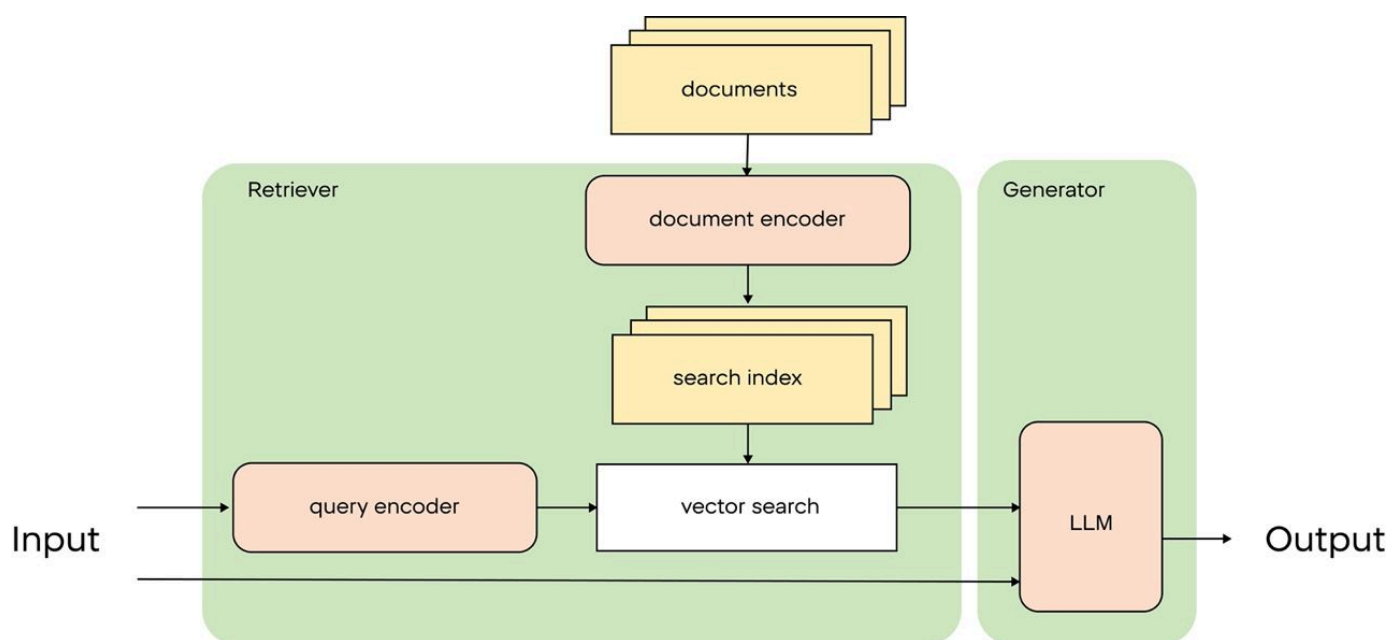


Схема работы RAG из оригинальной статьи

В качестве альтернативы подход RAG предлагает передавать в LLM релевантную вопросу пользователя информацию прямо внутри контекста. Например, если пользователь хочет

получить отчёт по продажам за март 2024 года, мы можем найти соответствующие документы, передать их в текстовом виде в контекст и использовать LLM для создания структурированного ответа. Этот подход значительно увеличивает полезность LLM в корпоративной среде, позволяя модели работать с конкретными и актуальными данными.

Отлично, но как же сделать это? Процесс построения простого RAG пайплайна состоит из трёх этапов:

1. *Преобразование документов*: внутренние документы (инструкции, нормативные документы, FAQ и т. д.) преобразуются в векторы с помощью моделей эмбедингов или индексируются, например, при помощи Elastic Search.
2. *Поиск документов*: при поступлении запроса пользователя он используется для поиска ближайших документов в базе данных.
3. *Генерация ответа*: LLM генерирует ответ на основе найденных документов и запроса пользователя.

► В виде кода это может выглядеть так

Но, к сожалению, простой RAG почти всегда не будет отвечать качеству поиска и генерации на ваших данных. Главным образом так получается потому, что стандартные подходы не всегда учитывают структуру данных: формат, необходимую длину контекста, распределение важной информации по документу и многое другое.

Поэтому наша работа здесь не закончилась, а только началась: мы долгое время экспериментировали с модификациями RAG-систем, и наконец готовы рассказать, что у нас вышло.

Улучшение RAG-системы. Retrieval

RAG состоит из двух частей: Retrieval и Augmented Generation. Эти части можно улучшать вместе (см. [hybrid rag](#)), но намного проще всё же по отдельности. В этой статье рассмотрим Retrieval, а вторую часть оставим для продолжения.

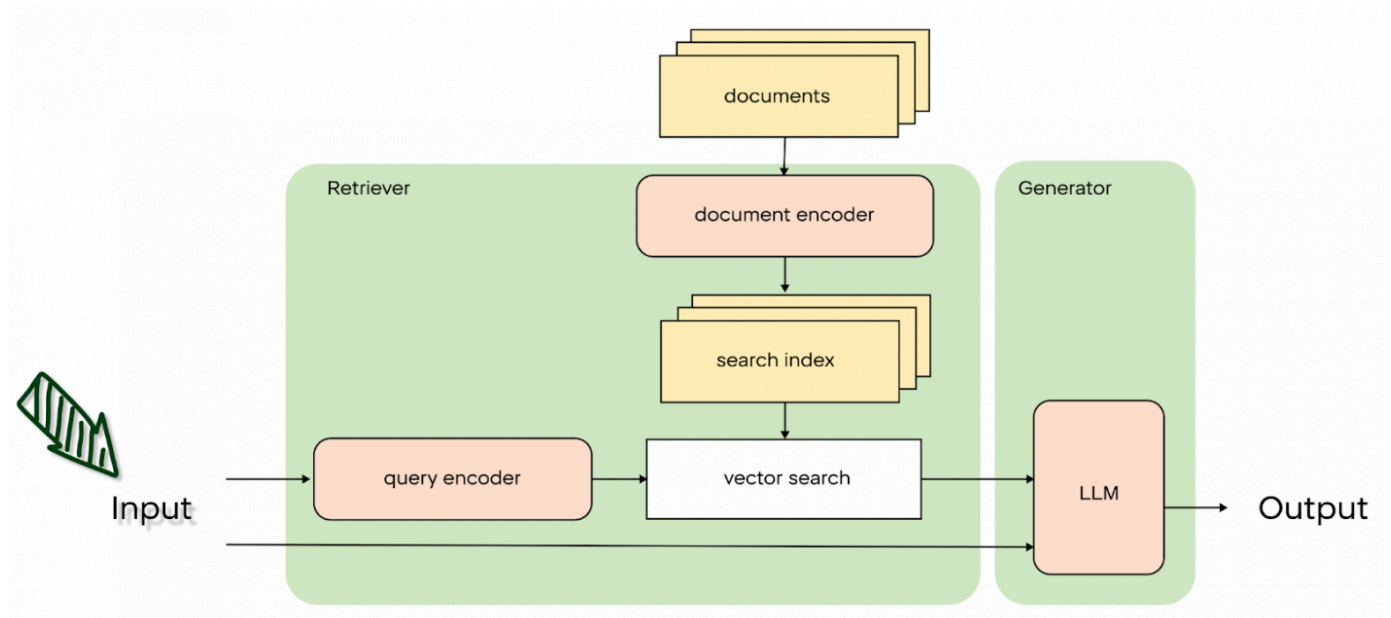
Мы хотим улучшать Retrieval, потому что хотим передавать как можно меньше нерелевантной информации в контекст модели – нерелевантная информация приведёт к лишним галлюцинациям, неправильным ответам, а если мы пользуемся коммерческими сетями (GPT, GigaChat, YaGPT), то передавая меньший контекст, мы ещё и сэкономим на токенах.

Retrieval-часть усложнённого пайплайна состоит из трёх этапов:

- обработка запроса пользователя (Query Expansion);
- поиск документов (контекстов) для ответа модели;
- переранжирование найденных документов для более точной выдачи.

Давайте рассмотрим, как мы улучшали каждую из этих частей.

Query Expansion



Работаем с исходным запросом пользователя

Query Expansion – это собирательное название техник, суть которых заключается в расширении и уточнении запроса пользователя. Эта техника помогает добавить больше контекста в исходный запрос пользователя, включая в него потенциальные ответы, которые помогают системе охватить более широкий спектр документов.

ЗАЯВКИ

2:00pm

Я не знаю как ответить на
ваш вопрос

2:01pm ✓

Как было "до"

Если исходный запрос был слишком неоднозначным, то добавление дополнительных ключевых слов или фраз, основанных на предполагаемых ответах, может значительно улучшить результаты поиска. Например, по запросу “отпуск” мы можем найти ответы на вопрос “оформить отпуск”, “перенести отпуск” и многие подобные.

Давайте рассмотрим примеры алгоритмов с самого простого до более сложных.

Перефразирование

Очень часто пользователи задают вопрос с опечатками или с минимальной информацией. Возьмём тот же пример, который мы уже использовали – запрос “отпуск”. Нам необходимо понять, что же хочет пользователь. Для этого мы можем сделать какое-нибудь предположение и не отвечать на вопрос “отпуск”, а перефразировать в “как оформить отпуск?”. Тогда вопросно-ответным системам гораздо легче будет ответить на такие вопросы. В идеале нужно уточнить у пользователя, что он имеет ввиду, но работа с историей чата – это тема отдельного поста. Основное, что нужно понять – это то, что наша LLM может помочь с перефразированием. Например, используя следующий промпт:

Ты – русскоязычный полезный ассистент. Ты помогаешь технической поддержке с поступающими вопросами. Пользователи не всегда пишут полные вопросы, ты всегда помогаешь перефразировать поступающий вопрос и выделяешь основную проблему.

Вот вопрос пользователя: “отпуск”

Перефразированный вопрос:

Ты - русскоязычный полезный ассистент. Ты помогаешь технической поддержке с поступающими вопросами. Пользователи не всегда пишут полные вопросы, ты всегда помогаешь перефразировать поступающий вопрос и выделяешь основную проблему.

Вот вопрос пользователя: “отпуск”
Перефразированный вопрос:

2:00pm

“Как оформить отпуск?”

2:01pm ✓

Как стало

Основной проблемой будет являться то, что модель будет некорректно перефразировать. Например, мы столкнулись с таким забавным перефразом:

Ты - русскоязычный полезный ассистент. Ты помогаешь технической поддержке с поступающими вопросами. Пользователи не всегда пишут полные вопросы, ты всегда помогаешь перефразировать поступающий вопрос и выделяешь основную проблему.

Вот вопрос пользователя: “Где найти план по выпечке?”
Перефразированный вопрос:

2:00pm

“Как приготовить пирог?”

2:01pm ✓

Некорректное преобразование запроса

Нужно постоянно следить и улучшать промпт, тогда система в целом будет отвечать лучше.

Мы используем разные промпты для разных наборов данных, чтобы избежать возникающих вопросов “про пироги”, а также предварительно оцениваем необходимость перефразы при помощи классификатора. Этот метод оказал существенное влияние на качество выдачи:

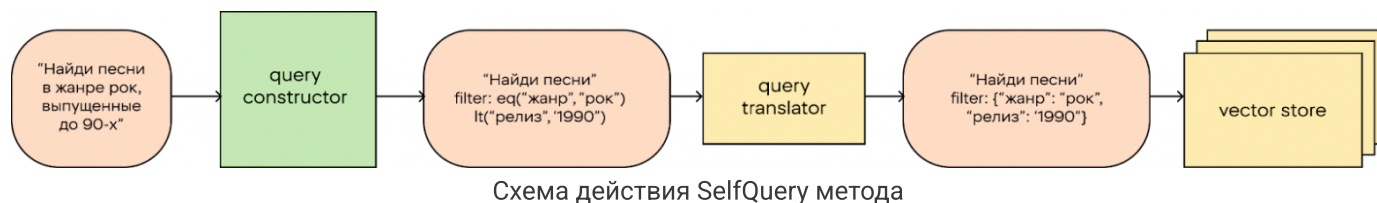
	MAP@5	p@1	p@5
Без перефразы	0.437	0.341	0.560
С перефразом	0.794	0.729	0.807

SelfQuery

Этот метод фокусируется на извлечении и использовании метаданных из самого запроса. Определение таких элементов, как географическая принадлежность, временные рамки

или специфическая терминология позволяет системе с большей точностью находить документы, наиболее релевантные поставленному вопросу, а выделение числовых значений из запроса позволяет напрямую использовать сравнение.

Схема работает так: мы вычлняем из запроса ключевые слова и значения при помощи промпта, по которым в дальнейшем можно будет проводить фильтрацию данных в базе.



Этот метод хорошо подходит, если запросы содержат достаточно подробную информацию, насыщены деталями и, особенно, цифры. Его большой плюс в том, что выбор метаданных производится при помощи нейросети, а потому будет работать даже при наличии опечаток и синонимов в запросе.

К сожалению, данный метод нам не подошёл, так как наши запросы содержат мало метаданных, и в целом достаточно короткие: в нашем же примере про отпуск метаданные выделяли категорию запроса (“отпуск”), а последующее переформулирование запроса (заложенное в промпт) приводило к тому, что полученный промпт получался пустым. Чисел, которые мы могли бы использовать для сравнения, в наших запросах тоже не было.

Вдобавок ко всему, если вы используете опенсорсную модель, она может плохо выделять метаданные и следовать шаблонам, что тоже является минусом данного метода.

Мы решили отказаться от использования SelfQuery, по крайней мере в имеющемся проекте, однако, этот метод выглядит крайне полезным для использования в проектах поиска недвижимости по заданным фильтрам.

FLARE (Forward-Looking Active Retrieval)

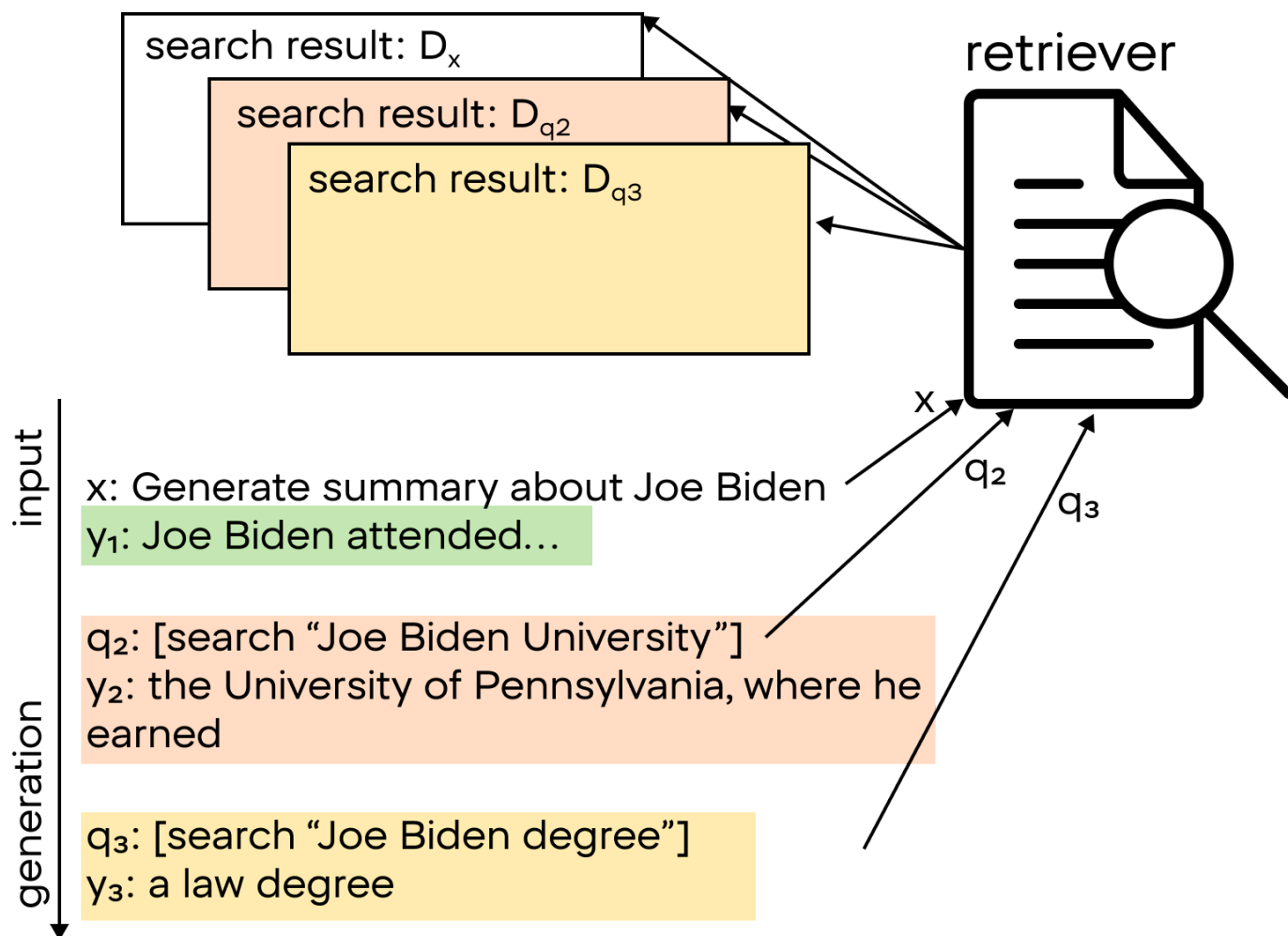


Схема работы метода FLARE

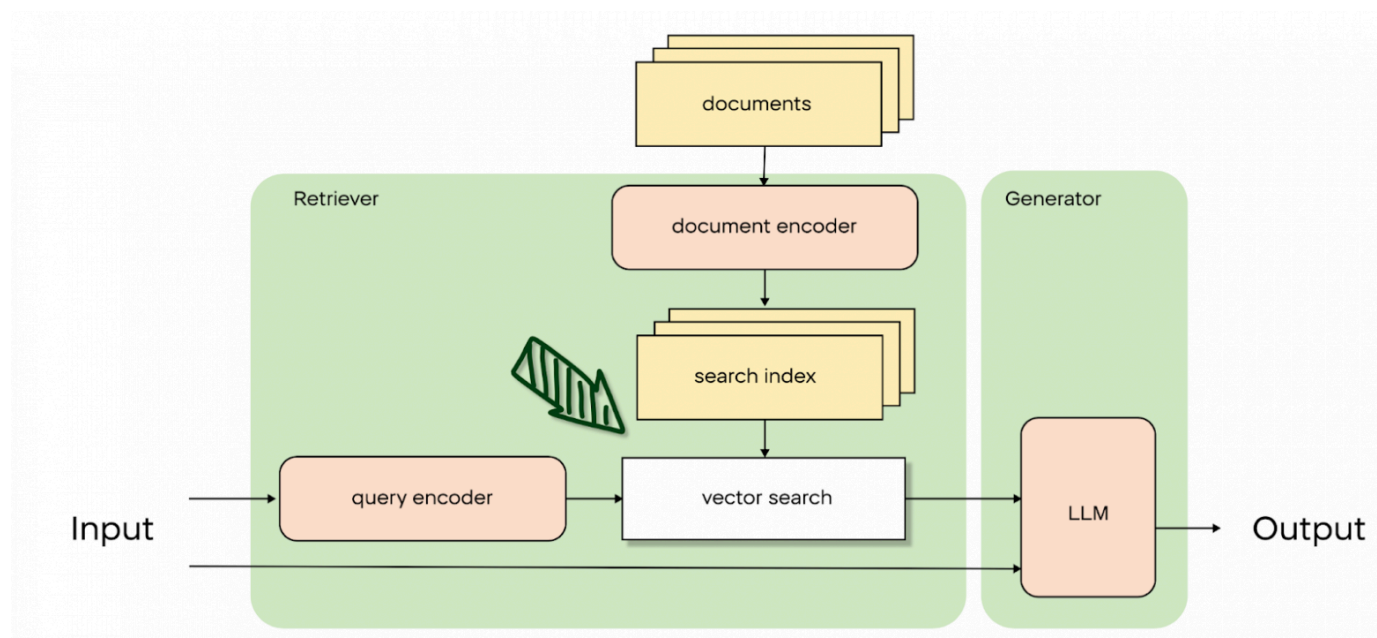
Эта техника включает в себя механизм повторного поиска, который активируется, если LLM не уверена в процессе генерации ответа. Уверенность модели мы измеряем при помощи распределения логитов функции потерь на выходе нейросети. Если проще, это "очки" ответов сети – если много ответов имеют похожие очки, то модель не уверена в своём ответе. Например, при генерации LLM ответ о биографии Байдена, если она начинает сомневаться, в каком университете он учился (это отслеживается, если вероятность токенов потенциального ответа снижается). Система отправляет запрос в LLM для уточнения места его учёбы, а дальше она продолжает генерировать ответ. Это продолжается до того момента, пока не будет сгенерирован символ остановки.

И хоть мы можем ограничить количество максимальных повторений цикла "поиск – генерация", при таком подходе мы тратим много времени и вычислительных мощностей на повторный поиск и приостановку генерации. Также модель имеет свойство далеко отходить от темы. В вопросах об отпуске она имела свойство размышлять о том, что такое отпуск, и не отвечать на заданный вопрос, поэтому на наших данных этот подход тоже было решено не использовать.

Данный метод будет хорош для использования в более творческих задачах, например, для написания статей, рассказов и эссе. Если ваша задача в бизнесе предполагает под собой какую-либо творческую свободу, этот метод может быть крайне полезным.

Методы Query Expansion, за исключением перефразирования, не стали применимыми в нашей задаче, однако в других они способны принести значительный результат. Поэтому вам всё же следует протестировать эти способы, а мы перейдём к другому этапу – Search Engine.

Search Engine

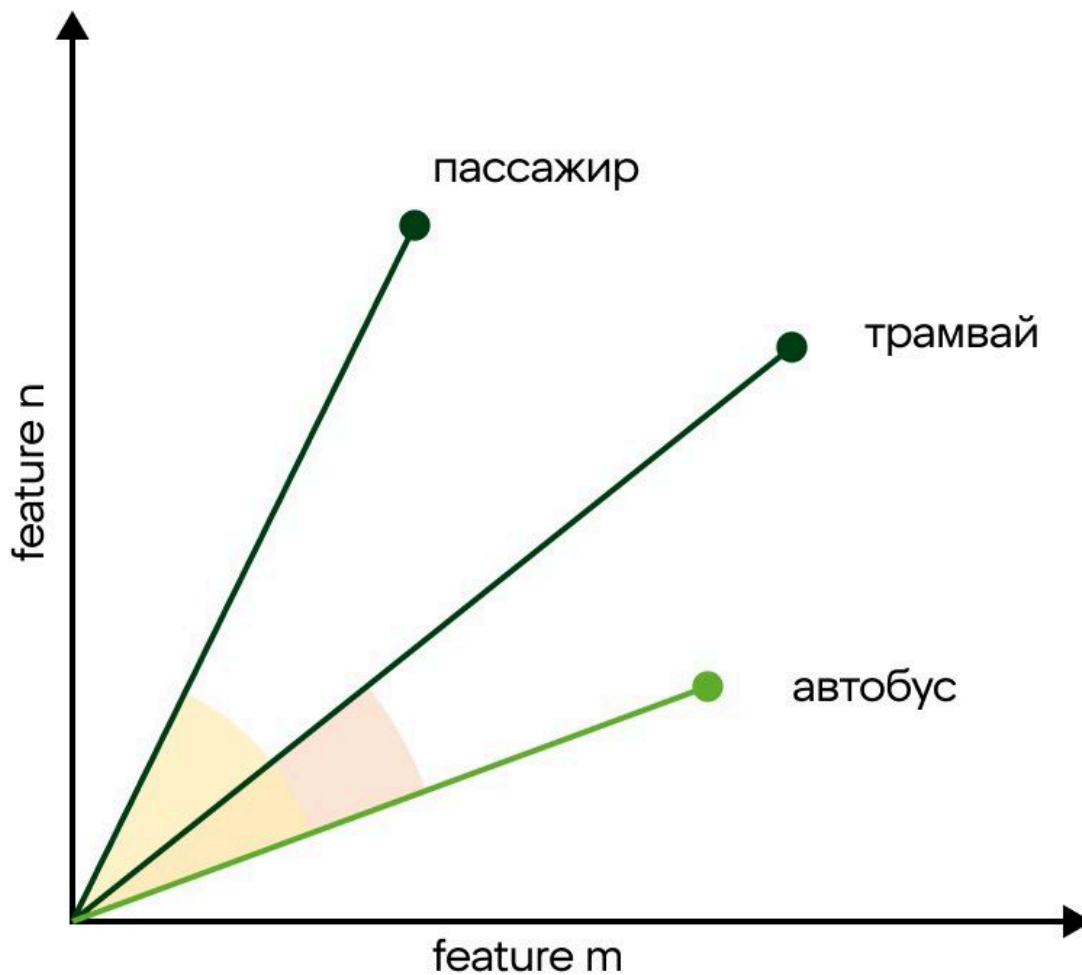


Теперь работаем с поиском

После обработки запроса пользователя начинается первичный поиск подходящих документов. Здесь можно сделать параллель с поисковыми движками, например, Google или Yandex. Но в нашем случае нужно искать информацию на наших внутренних данных, а не во всём интернете. Давайте рассмотрим, как мы можем искать подходящие под запрос документы.

Векторный поиск

Самый популярный сейчас способ – векторный поиск. Это часто называют семантическим поиском. Суть метода заключается в том, что каждый текст можно представить в виде вектора – набора чисел. Векторы можно сравнивать друг с другом, как смыслы текстов. Чем меньше расстояние между векторами – тем более похожи по смыслу сравниваемые тексты. Для сравнения векторов используется косинусное расстояние. Поэтому давайте найдём векторы всех внутренних документов и будем искать наиболее близкие по конусному расстоянию документы к запросу пользователя. Таким образом и появляется идея векторного поиска.



Косинусное расстояние

Сейчас уже есть очень много специализированных векторных баз данных, мы используем у себя Qdrant. Эта технология позволяет с коробки сделать то, что мы описали выше.

В векторном поиске главным предметом для экспериментов и улучшения качества поиска является способ представить текст в виде векторов. Есть огромное количество методов, их качество на ваших данных будет зависеть от языка, средней длины и домена. В своей работе мы пробовали эмбединги clips/mfaq, intfloat/multilingual-e5-large и эмбединги от LLaMA. Самый лучшим инструментом векторизации у нас оказался multilingual-e5-large, однако вы можете держать руку на пульсе, просматривая лидерборд [здесь](#), выбирая эмбедек под задачу, релевантную для своего проекта.

Другие варианты

На самом деле, есть очень много способов сравнить два текста с друг другом. И даже не обязательно использовать что-то одно, их можно комбинировать и объединять в ансамблевый ретривер.

BM25 и полнотекстовый поиск

Использование векторного поиска не означает, что мы не можем использовать поиск по ключевым словам – зачастую (в особенности при поиске документов с различными аббревиатурами и терминами) бывает полезным использовать информацию о словах, встречающихся в документе.

Реализацию BM25 и подробности можно посмотреть, например, [здесь](#).

Симметричный поиск

Идея данного подхода проста, и при этом он работает очень хорошо: мы ищем не документы, релевантные запросу, а другие вопросы, похожие на этот. Их мы достаём из базы данных, которая легко пополняется на всём протяжении работы сервиса при помощи статистик вопросов пользователей.

Каждый вопрос в базе данных привязан к конкретному документу – именно эти документы мы передаём в качестве результата симметричного поиска. Таким образом мы значительно повышаем качество поиска релевантного контекста на часто встречающихся вопросах.

Если идти дальше, то можно определять степень “частоты” запроса (например, измеряя его расстояние до остальных вопросов в базе) и, если этот вопрос всё же окажется редким, то уже запускать другие алгоритмы поиска.

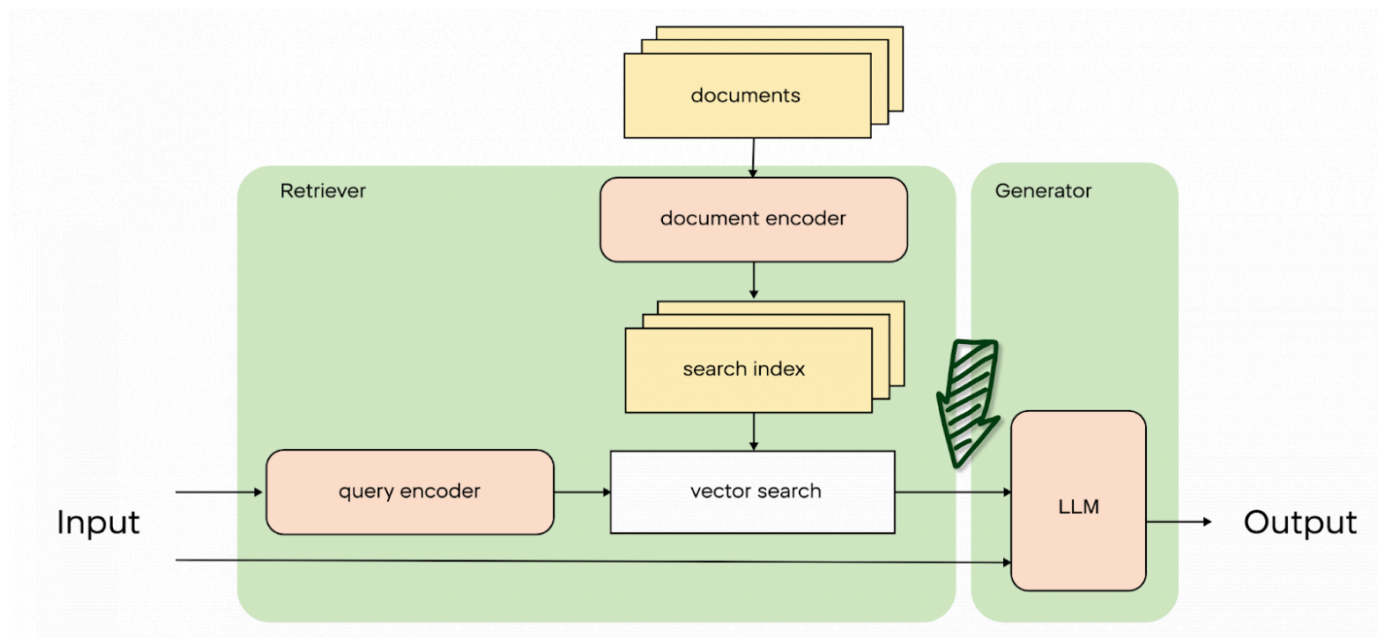
SVM

Для ранжирования документов на основе их релевантности запросу можно также использовать SVM. Для этого мы трансформируем задачу ранжирования в задачу классификации: релевантные документы отмечаем, как положительный класс, а нерелевантные – как отрицательный. Таким образом мы используем SVM-классификатор для разделения документов на нужные нам и посторонние.

Для наших же данных лучше всего себя показал ансамбль из векторного поиска + SVM ретривера.

Вы же для своих данных можете попробовать разные варианты, [библиотека langchain](#) предоставляет очень много вариантов.

Post Processing



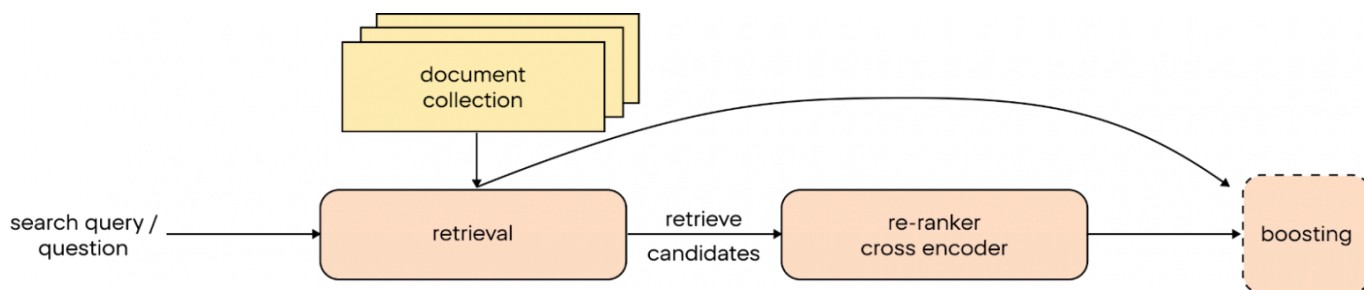
Теперь работаем с обработкой найденных документов

Финальной стадией работы Retriever в RAG-системах является постобработка. На этом этапе происходит пересмотр и перераспределение найденных документов так, чтобы наиболее релевантные запросу пользователя находились на первых позициях.

Первоначальное ранжирование документов, выполненное ретривером, может быть неидеальным. Это связано с использованием упрощённых алгоритмов, которые не всегда учитывают контекст запроса. В результате релевантные документы могут быть не на первых позициях, что снижает эффективность поиска. Напомню, что мы можем передать ограниченное количество документов в наш LLM из-за ограничения размера контекста модели.

Для улучшения точности ранжирования документов используются более сложные методы.

Cross-encoder



Механизм работы постобработчика

Это нейросеть, обученная оценивать степень близости текстов друг с другом. Казалось бы, в чём отличие от нашего семантического поиска? Огромное – Cross-Encoder более сложный алгоритм, который отлавливает глубокие смысловые значения в текстах. В процессе работы он сравнивает каждый найденный документ с запросом пользователя, выдавая численную оценку схожести. Теперь можно повторно отсортировать документы по новой оценке.

Это также позволило значительно повысить значения целевых метрик:

	MAP@5	p@1	p@5
До переранжирования	0.794	0.729	0.807
После переранжирования	0.904	0.835	0.972

Но что, если мы хотим сортировать не только по схожести от кросс-энкодера, но и от других?

Boosting

Используем классический ML. Например, мы можем обучить градиентный бустинг. Этот метод объединяет множество слабых моделей для создания сильного прогнозирующего алгоритма. В нашем контексте градиентный бустинг может использоваться для более точного определения релевантности каждого документа по отношению к запросу на основе любых фичей: схожесть от кросс-энкодера, BM25 и другие скоры.

Постобработка является ключевым этапом в работе RAG-систем, так как она позволяет существенно повысить качество поиска за счёт более точного ранжирования документов. Применение методов, таких как Cross-Encoder и градиентный бустинг, обеспечивает более контекстуальное и релевантное расположение документов.

В заключение

На этом у нас всё. В этот раз мы рассмотрели несколько вариантов того, как мы улучшали поиск релевантных документов для RAG-систем:

- Первичную обработку запроса пользователя при помощи методов Query Expansion: перефразирования вопроса, выделения метаданных и поддержания уверенности нейросети в своих генерациях.
- Векторный поиск документов в базе данных.
- Последующее переранжирование найденных документов.

В следующих статьях расскажем про улучшение генераций на основе уже найденных документов, как мы мониторим LLM в проде, боремся с галлюцинациями и многое другое.

Авторы: Мичил Егоров , Глеб Панин , Дарья Андреева

Теги: llm, rag, ai, чат-бот, искусственный интеллект, ии, ии-ассистент, python, retrieval, retrieval augmented generation

Хабы: Блог компании X5 Tech, Python, Искусственный интеллект, IT-компании

Редакторский дайджест

Присылаем лучшие статьи раз в месяц

Электронпочта

Подписаться

Оставляя почту, я принимаю Политику конфиденциальности и даю согласие на получение рассылок



32K+

Охват за 30 дней

X5 Tech

Всё о технологиях в ритейле

[Сайт](#) [Сайт](#) [ВКонтакте](#) [Telegram](#)



8

Карма

Дарья Андреева @xufana

LLM Researcher, научный сотрудник

Подписаться



Комментарии 6

Публикации



sound_right
6 часов назад

Нам нужен сотрудник с горящими глазами

Простой 9 мин 16K

Аналитика

+42

36

36



PatientZero
20 часов назад

Структуры данных на практике. Глава 10: В-деревья и деревья, эффективно использующие кэш

Простой 9 мин 13K

Тutorial

Перевод

+30

95

5



TrexSelectel
5 часов назад

Парализованный мужчина управляет World of Warcraft мыслью: сто дней с нейроинтерфейсом Neuralink

4 мин 4.8K

+26

2

2



shvedov_grangroup
2 часа назад

Дефект в 8 микрон прошёл все проверки и отправил в брак тысячи готовых плат. Рассказываю, как мы его нашли

Простой 8 мин 4K

Кейс

+25

6

1



tapeline
18 часов назад

Парсер-комбинаторы «с нуля»



Средний



19 мин



12K

Тutorial



+25



69



9



Catx2

23 часа назад

Белый раст. Сердце Московского энергетического кольца



6 мин



13K



+25



13



5



RationalAnswer

8 часов назад

Кибероттепель в Москве, а также слухи о грядущих AGI-моделях от OpenAI & Anthropic



8 мин



8.9K

Дайджест



+22



3



9



40oleg

22 часа назад

Машина Тьюринга в Minecraft



Средний



4 мин



12K

Кейс



+21



17



3



Kotelnikovekb

23 часа назад

Чиним доступ к Telegram, GPT и другим API из России



Простой



3 мин



45K

Тutorial



+19



123



33



alizer
4 часа назад

Ненужные технологии



Простой



7 мин



4.8K

Мнение



+17



4



14

Показать еще

ВАКАНСИИ КОМПАНИИ «X5 TECH»

Python разработчик (Middle+ / Senior)

X5 Tech · Москва · Можно удаленно

Разработчик/Старший разработчик 1C

X5 Tech · Москва

Больше вакансий на Хабр Карьере



Настройка языка

Техническая поддержка

© 2006–2026, Habr